

Who is responsible for an executable claim? Epistemic accountability in agent-assisted research

Abstract

An executable answer makes a model's computation inspectable, but does not establish that the computation answers the intended question. This distinction becomes an ethical issue when coding agents also help construct the data, tests, and scientific explanation. We develop an account of epistemic accountability through a bounded case of small-model research. The case system, SOP Lang, represents a solution as named computations with explicit dependencies and executes them in a runtime. We explain its notation and use a constructed counterexample to show how a generated program and reference solver can agree on the same mistaken interpretation. Archived experiments also supply a constructive finding: specialized commands increase normalized exact matching from 53.8% to 62.4%, while a different change that adds intermediate structure increases execution failures. Our argument is that the benefit of delegation creates duties to specify what was checked, preserve the evidence needed to challenge a claim, and disclose uncertainty where it changes the inference. These duties remain with human authors even when agents produce much of the implementation and analysis. The case does not measure human overtrust or compare agents with human research assistants. It supports a practical standard for developing new abstractions that improve computation while keeping their assumptions open to scrutiny.

Keywords: epistemic accountability; artificial intelligence; scientific practice; coding agents; executable reasoning; small language models

1. Introduction

A program is persuasive evidence of something. It can show the operations that produced an answer and allow another person to execute them again. The difficult question is what that evidence justifies. A reproducible calculation may still implement the wrong interpretation of a problem.

This article examines that question in research conducted with coding agents. Its case is SOP Lang, a language developed to let small models translate word problems into explicit executable programs. The model supplies the quantities and selected operations; a runtime carries them out. We introduce the notation in Section 2 because the argument depends on seeing exactly what becomes explicit and what remains assumed.

Agents can participate in more than the final program. They may help build the training examples, reference computations, tests, experiment scripts, and manuscript. Agreement among those artifacts can be useful, but can also arise because an assumption has been copied across them. The ethical issue is whether the published claim gives readers grounds proportionate to the confidence it invites.

Messeri and Crockett describe how AI can create illusions of understanding in scientific research [1]. We examine one concrete pathway to that concern: executable agreement may be mistaken for independent justification. We also preserve the positive case for delegation. Tested abstractions can help small models perform useful computations, and an inspectable program can make some mistakes easier to challenge.

The research question is what makes computational delegation a justified source of scientific evidence when agents also help construct its checks and interpretation. We argue that epistemic accountability requires a reader to be able to distinguish the observed result, the property checked, and the explanation proposed. Human authors remain responsible for the relation between those three. This is a normative argument grounded in an artifact case, not a psychological study of overtrust or a measurement of research-assistant productivity.

2. What an executable answer exposes

2.1 The case language, without assumed prior knowledge

SOP Lang represents a program as a circuit of named computations called wires. A declaration starts with `@name` command at the beginning of a line. Its body continues until the next declaration. The command determines the body's meaning: `literal` reads JSON, while `jsEval` evaluates JavaScript. A reference beginning with a dollar sign reads another wire's value and declares a dependency.

For "three packs containing four cells each," the following complete circuit returns 12:

```
@slots literal
{"packs":3,"perPack":4}

@answer jsEval
return $slots.packs * $slots.perPack;
```

The names `slots` and `answer` identify the two computations. They are conventions used by the dataset, not reserved keywords. The `literal` command supplies an object; `$slots.packs` reads its `packs` field. JavaScript's `return` supplies the value of the second wire. The caller requests the output named `answer`. The runtime executes dependencies first, so textual order alone does not determine evaluation.

Figure 1 shows what the program exposes. A reader can check that the model chose multiplication, extracted 3 and 4, and produced 12. The same reader must still compare that choice with the question. If the task asked for usable cells after one damaged cell per pack, this executable multiplication would be inadequate.

A complete compiled solution

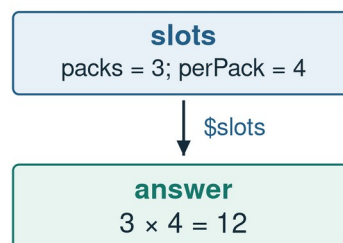
Problem: three packs, four cells per pack. How many cells?

GENERATED PROGRAM

```
@slots literal
{"packs":3,"perPack":4}

@answer jsEval
return $slots.packs * $slots.perPack;
```

VALUE DEPENDENCY



`@name` command starts a wire; its body follows on the next line.

`$name` reads a value. The reference determines execution order.

Fig. 1 An executable answer makes a particular interpretation inspectable. The example is constructed and executed for explanation; it is not a new experimental observation

The system also supplies specialized operations. A graph-reachability command, for example, takes an explicit edge list and endpoints and performs traversal internally. It removes implementation steps from the generated program. It does not remove the need to decide whether the problem concerns directed or undirected links, reachability or shortest paths, and the intended endpoints.

2.2 Execution is a reason for a bounded belief

PAL and Program of Thoughts demonstrate how language models can delegate computation to interpreters [2] [3]. The benefit is concrete. Arithmetic need not be repeated through token prediction, and the operations leading to a result can be inspected. That supplies a reason to trust the execution of a stated program when the runtime's behavior has been adequately tested.

The reason remains conditional on the specification. Gulwani, Polozov, and Singh treat the specification as a central problem of program synthesis [4]. In natural-language compilation, specification includes which quantities matter, what their relations are, and which outputs count as answering the question. A runtime cannot resolve all those choices by executing the program they produced.

We use epistemic accountability to mean that the grounds and limits of a scientific claim remain available for challenge, and that identifiable human authors accept responsibility for its interpretation and publication. This does not require a person to recompute every arithmetic step. It requires an honest account of what the automated step establishes.

2.3 Abstraction also chooses what to exclude

A specialized command can help by hiding implementation details. The model need not generate a traversal queue or fraction-reduction algorithm. Yet the operation also embodies choices about admissible inputs and relevant distinctions. An undirected reachability command excludes edge direction from its semantics. That is appropriate for some problems and wrong for others.

Selbst and colleagues explain how abstraction in sociotechnical systems can omit context important to the problem being addressed [5]. Our computational case is narrower, but the connection is precise: successful operation inside a formal description does not establish that the description retained the relevant distinctions. We do not transfer the word-problem results to fairness or public decision-making applications. Their relevance is to how a claim should expose the assumptions of its chosen abstraction.

DreamCoder offers a computational motivation for discovering reusable program abstractions [6]. The present commands were developed through human-directed work assisted by coding agents. Their scientific value depends on improving the task while preserving enough visibility to assess when the command should be used.

3. Evidence and analytical basis

3.1 The experimental case

The project constructs training examples from parameterized problem families with reference parses, answer computations, and generated circuits. It executes candidate circuits, compares answers, asserts selected properties, and perturbs literal inputs to detect some forms of hard-coding. These are substantial attempts at data rigor. They do not make the parser, reference computation, and circuit semantically independent when all originate in the same family definition.

The primary evidence reconstruction covers ten archived fine-tuning runs, each evaluated on 705 items. It reproduces the original normalized exact classifications, resolves model identity from manifests, compares task populations, and inspects generated commands. Targeted checks also revisit base-model evaluations. No new training, model generation, or participant study is performed.

Normalized exact matching means equality after stated text normalization, not general semantic correctness. The benchmark is synthetic, contains repeated computational plans, and was inspected during development. Dwork and colleagues' analysis of adaptive reuse explains why such a benchmark cannot provide independent confirmation of every later design choice [7]. There is one run per condition in the comparisons discussed here.

3.2 A useful result that deserves to be retained

The specialized-command comparison uses the same Qwen2.5-Coder-1.5B base release in both conditions. Introducing graph, aggregation, and fraction operations increases normalized exact matching from 53.8% to 62.4%. Procedural execution failures decline from 13.1% to 2.7%. The full evaluation contains 705 cases per condition; the procedural subset contains 480.

This result supports taking the model-runtime interface seriously as a research variable. Our interpretation is that moving recurring algorithms into tested commands can reduce the implementation burden on a small model. The comparison lacks replicated seeds and a complete early training manifest, so it does not establish a general causal law. Its positive contribution remains worth reporting within that boundary.

A separate representation change supplies a constraint on the interpretation. Splitting computations into more intermediate wires increases execution failures from 8.1% to 19.1% on 705 problems per condition. We think the added interfaces can impose coordination work without removing enough algorithm construction. The archive does not classify every failure mechanism, but the outcome already rules out the simple recommendation to add structure indiscriminately.

The ethical significance is that the evidence can improve the project's own preferred story. Successful abstraction and unsuccessful decomposition belong in the same account of what researchers should investigate next. Preserving the negative result serves the reader who might otherwise invest in an attractive but unsupported design heuristic.

3.3 Corrected interpretations as bounded examples

One historical run label suggested a 17B model, while its manifest identifies a 1.7B release. The records support the latter identity. A claim that a much smaller model beat a tenfold larger one would therefore be misleading even if every reported outcome count were correct. The error lies in what the comparison is said to mean.

Another claimed semantic summary cannot be reconstructed because its individual judge verdicts are absent. Saved requests show what a judge was asked; they do not show what it decided. We exclude the aggregate as evidence. This is a duty to preserve the difference between missing support and disproven support: the absence of decisions does not establish that every original decision was false.

These examples are specific to the surviving archive. They do not show that agents are uniquely prone to such errors or that human research assistants would avoid them. They show why the responsibilities of publication cannot be satisfied by internal consistency alone.

4. The independence problem

Figure 2 gives a constructed counterexample. The stated graph has directed links $B \rightarrow A$ and $C \rightarrow B$. A cannot reach C by following those directions. Suppose a parser discards direction and supplies an undirected edge list to both the generated solver and a reference computation. Both return *yes*. Their agreement is reproducible, but it supports the wrong interpretation of the original problem.

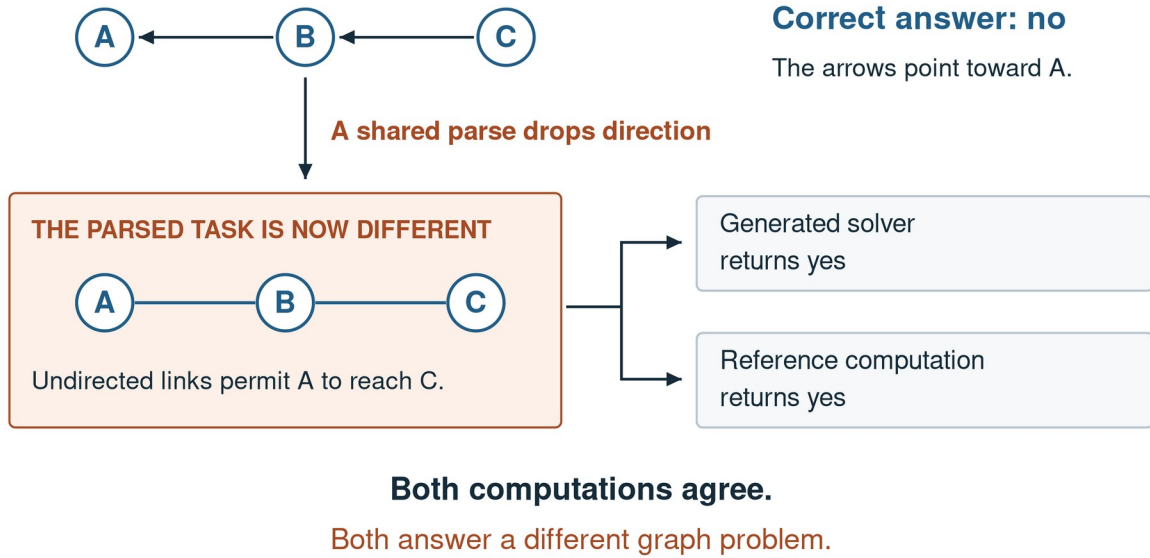
Constructed counterexample: agreement with a shared mistake**STATED TASK:** directed links; can A reach C?

Fig. 2 Constructed counterexample to treating agreement as independence. The source condition is lost before either computation checks the result. The diagram is explanatory and does not assert an observed error frequency

The example explains why more tests are sometimes insufficient. Tests of two implementations against the same parsed graph can increase execution coverage without challenging the omitted direction. The additional evidence must address the relation between the source statement and the parse, for example through an independently constructed counterexample or targeted human review.

Raji and colleagues describe auditing as a documented process across an AI development lifecycle [8]. In research, that process must preserve the distinction between construction, verification, and interpretation. Table 1 states the corresponding limits of the evidence.

Table 1. What different checks warrant, and the question that remains for the author.

Check	Warranted belief	Remaining question
Program parses and its dependencies are valid	The program meets structural rules	Does it express the intended task?
Runtime returns a value	The selected computation completes under the runtime contract	Are the selected operation and arguments appropriate?
Program and reference agree	Two results agree under the declared comparison	Do their implementations share the same mistaken premise?
Perturbation changes the answer	The output depends on the tested inputs	Were the right inputs and constraints extracted?
Saved labels reproduce a percentage	The aggregate follows from those labels	Do the labels measure the property claimed?

This table does not diminish the value of checks. It identifies the belief for which each check supplies a reason. Calling all five operations "verification" without naming their object invites a stronger inference than any of them warrants.

5. Duties that follow from delegation

5.1 Describe the checked property accurately

A scientific author should state what the check tested. "The saved comparison labels reproduce" is a clear claim. "The reasoning was verified" may suggest that task interpretation, execution, and judgment were all independently established. The present archive cannot support that broader statement.

The same requirement applies to training-data rigor. Execution against reference answers and input perturbations are meaningful engineering controls. They should be described as those controls, with their shared assumptions identified. A reader can then decide whether they are sufficient for a proposed reuse or whether an independent review of problem meaning is needed.

This obligation follows from the reader's dependence on the author's description. Readers cannot reconstruct every pipeline before deciding which results deserve attention. Overstating a check transfers an undisclosed verification burden to them and can misdirect later research.

5.2 Preserve the evidence needed to disagree

A result becomes more accountable when a reader can challenge model identity, population continuity, scoring, and mechanism separately. Source records, exact model releases, generated programs, and item-level decisions make those challenges possible. A summary table alone often does not.

Sandve and colleagues' reproducibility guidance supports retaining executable transformations and their inputs [9]. Datasheets and model cards similarly motivate documentation of construction, intended use, and evaluation conditions [10] [11]. In an agent-assisted workflow, this documentation should also state which components share a parser or reference implementation. Independence is a relation between sources of evidence, not a property obtained by storing them in different files.

This duty applies to failures as well as successes. A missing model response should remain visible as a response-availability problem. A rejected answer outside a diagnostic grammar should remain unclassified. An absent judge decision should not be filled with a plausible answer to rescue a headline.

5.3 Attach uncertainty to the inference it changes

A general limitations section cannot repair every strong claim earlier in a paper. The missing early training manifest belongs beside the abstraction result because it limits causal attribution. A changed problem population belongs beside the comparison it affects. A weak number-presence scorer belongs beside the reported percentage because it determines what that percentage measures.

The required practice is proportionate. Exploratory work need not complete every future control before publication. It must present an exploratory result as such and make the next discriminating experiment clear. The positive abstraction result justifies further research; it does not justify a universal scaling law or deployment claim.

This distinction also avoids an unproductive response to imperfect evidence. A useful developmental finding should not disappear merely because an older interpretation was too ambitious. Narrowing the inference preserves scientific value while making disagreement more precise.

5.4 Retain human responsibility for the published claim

An agent can generate code, compare records, or propose an interpretation. It cannot assume the author's responsibility to decide which claim is justified, which uncertainty matters, and which correction must be published. That responsibility concerns the use of the evidence, not merely ownership of the final text.

Disclosure should identify the affected stages. In this case agents assisted implementation, training-data tooling, analysis, and writing, including the present audit. Describing that involvement as grammar correction would conceal a material condition of the research. Conversely, treating agent participation as automatic disqualification would ignore the executable work that can be inspected.

Human responsibility does not mean claiming perfect oversight. Authors should state what they checked, what remains unresolved, and how the evidence can be challenged. The same standard applies to this article; agent-assisted reconstruction reduces some mistakes but does not make the interpretation independent or infallible.

6. Discovering new wires without hiding their assumptions

The positive result opens a concrete research direction. A dependency-join command could own scheduling over stated predecessor relations. A units-and-rates command could check dimensional compatibility before calculation. Such operations could reduce recurring implementation failures while keeping the extracted data visible.

The ethical and technical tests align here. A candidate should state its input assumptions, exclusions, implementation, and adversarial cases. A controlled comparison should evaluate equivalent general-code and specialized-command targets across several seeds, with transfer tasks frozen before design. Reporting should include wrong-command selection and wrong-argument extraction, not only execution success.

A shorter program is not sufficient evidence of a better research abstraction. If the command hides a disputed classification or silently discards a relevant condition, it can make generation easier while making the claim harder to examine. A useful wire exposes the information needed to determine when its result applies.

No social-deployment, energy, or equitable-access benefit is measured by this case. Those may motivate future applications, but would require their own evidence. The contribution here is a standard for the research process: make computational delegation easier to inspect at the same time that it becomes easier to perform.

7. Conclusion

Executable programs can improve scientific evidence by exposing the computation behind an answer. The SOP Lang case shows a real benefit from specialized operations and a real limit to adding structure. It also shows why reproducible outputs do not settle model identity, task interpretation, or the meaning of an evaluation score.

Our conclusion is that delegated computation creates obligations to specify the checked property, preserve the grounds for challenge, and keep uncertainty attached to the affected inference. Human authors remain responsible for those obligations. The next abstraction research should seek commands that reduce implementation errors while making their assumptions and remaining interpretation choices clear.

Data availability and use of AI-assisted tools

Online Resource 1 contains reconstructed outcomes, source mappings, comparison records, diagnostic decisions, and analysis scripts. An anonymized stable access location is required before double-anonymous submission. This manuscript omits identifying author metadata and repository URLs. Coding-agent assistance is described in Sections 3 and 5.4. Author identities, funding, competing interests, and acknowledgements require separate completion.

References

- [1] Messeri, L., & Crockett, M. J.: Artificial intelligence and illusions of understanding in scientific research. *Nature*, 627(8002), 49-58 (2024). <https://doi.org/10.1038/s41586-024-07146-0>
- [2] Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., & Neubig, G.: PAL: Program-aided Language Models. *Proceedings of Machine Learning Research*, 202, 10764-10799 (2023). <https://proceedings.mlr.press/v202/gao23f.html>
- [3] Chen, W., Ma, X., Wang, X., & Cohen, W. W.: Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research* (2023). <https://openreview.net/forum?id=YfZ4ZPt8zd>
- [4] Gulwani, S., Polozov, O., & Singh, R.: Program Synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2), 1-119 (2017). <https://doi.org/10.1561/25000000010>
- [5] Selbst, A. D., Boyd, D., Friedler, S. A., Venkatasubramanian, S., & Vertesi, J.: Fairness and Abstraction in Sociotechnical Systems. *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 59-68 (2019). <https://doi.org/10.1145/3287560.3287598>
- [6] Ellis, K., Wong, C., Nye, M., Sablé-Meyer, M., Morales, L., Hewitt, L., Cary, L., Solar-Lezama, A., & Tenenbaum, J. B.: DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning. *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 835-850 (2021). <https://doi.org/10.1145/3453483.3454080>
- [7] Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., & Roth, A.: The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248), 636-638 (2015). <https://doi.org/10.1126/science.aaa9375>
- [8] Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., & Barnes, P.: Closing the AI accountability gap: defining an end-to-end framework for internal algorithmic auditing. *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 33-44 (2020). <https://doi.org/10.1145/3351095.3372873>
- [9] Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E.: Ten Simple Rules for Reproducible Computational Research. *PLoS Computational Biology*, 9(10), e1003285 (2013). <https://doi.org/10.1371/journal.pcbi.1003285>
- [10] Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé, H., III, & Crawford, K.: Datasheets for datasets. *Communications of the ACM*, 64(12), 86-92 (2021). <https://doi.org/10.1145/3458723>
- [11] Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T.: Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220-229 (2019). <https://doi.org/10.1145/3287560.3287596>